



Deep Learning

*Projet fin de Module
Secteur financier*

Année : 2025-2026

Réalisé par

Ismail FAIDA

Encadré par

Mme Zineb HIDILA

Remerciements

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin de module.

Mes remerciements les plus chaleureux vont en premier lieu à Madame Zineb Hidila, professeure responsable du module Deep Learning à l'EMSI Casablanca, pour la qualité exceptionnelle de son enseignement, sa disponibilité constante et les précieux conseils prodigués tout au long de ce module. Son approche pédagogique alliant rigueur théorique et application pratique a constitué le socle sur lequel repose l'intégralité de ce travail. La richesse des concepts abordés en cours – des fondements du perceptron multicouche aux architectures récurrentes avancées – m'a permis d'appréhender le deep learning avec la profondeur nécessaire à la réalisation de ce projet.

Je remercie également l'ensemble du corps professoral du département Informatique de l'EMSI Casablanca pour la formation d'excellence dispensée tout au long de ces quatre années d'études en Ingénierie Informatique, spécialité Intelligence Artificielle et Data Science (IAD & DSI).

Enfin, je remercie mes camarades de promotion pour les échanges intellectuels stimulants, ainsi que la communauté open source PyTorch, Kaggle et HuggingFace dont les ressources librement accessibles ont rendu possible l'expérimentation sur des données réelles de qualité.

Table des matières

Remerciements.....	2
Résume	5
Abstract	6
Introduction	7
1. Contexte général	7
2. Problématique	7
3. Organisation du projet	7
4. Méthodologie générale	8
5. Choix du thème financier	8
Partie I – MLP et Ingénierie PyTorch.....	9
Détection de fraudes par carte bancaire	9
1.1 Contexte métier et problématique	9
1.2 Description du dataset	9
1.3 Fondements théoriques du MLP	9
1.4 Concepts PyTorch fondamentaux	10
1.5 Préparation des données	10
1.6 Les deux implémentations du MLP	10
1.7 Stratégies d'initialisation.....	11
1.8 Boucle d'entrainement et choix des hyperparamètres	11
1.9 Métriques d'évaluation	12
1.10 Résultats expérimentaux.....	12
1.11 Question de synthèse – Partie I	13
Partie II – CNN et Vision par Ordinateur	14
Classification de patterns techniques sur graphiques boursiers.....	14
2.1 Contexte métier et problématique	14
2.2 Description du dataset	14
2.3 Pourquoi un MLP est inadapté aux images de graphiques.....	14
2.4 Les trois idées fondatrices du CNN	15
2.5 Calculs dimensionnels	15
2.6 Architecture CNN – justification de chaque choix	15
2.7 Implémentation manuelle des opérations convolutionnelles	16
2.8 Visualisation des cartes de caractéristiques	16
2.9 Comparaison CNN vs MLP	16
2.10 Question de synthèse – Partie II	17
Partie III – RNN, LSTM, GRU et Seq2Seq	18
Génération de news financières	18
3.1 Contexte métier et problématique	18
3.2 Fondements théoriques	18

3.3 Comparaison RNN, LSTM et GRU.....	19
3.4 Preparation des donnees textuelles	19
3.5 Architecture Seq2Seq	19
3.6 Stratégies de décodage	20
3.7 Résultats expérimentaux.....	20
3.8 Question de synthèse – Partie III	20
4.1 La geometrie des donnees dicte l'architecture	21
4.2 MLP : le modèle universel sans Prior	21
4.3 CNN : exploiter la localité spatiale.....	21
4.4 RNN/LSTM/GRU : modéliser la temporalité	21
4.5 Paradigme unifié, déclinaison architecturale	21
4.6 Perspectives en finance quantitative	22
Conclusion	23
Bilan méthodologique	23
Bilan expérimental.....	23
Apports et limites.....	24
Perspectives.....	24
Annexe – Références et bibliographie.....	25
Architectures et fondements théoriques	25
Datasets et outils	25
Contexte réglementaire.....	25

Résumé

Ce projet de fin de module explore l'application du deep learning au domaine de la finance à travers trois architectures complémentaires. La Partie I implémente un Perceptron Multicouche (MLP) pour la détection de fraudes par carte bancaire sur le dataset Credit Card Fraud Detection (284 807 transactions, 0,17% de fraudes). La gestion du déséquilibre de classes via la pondération `pos_weight`, l'initialisation Xavier et la régularisation par BatchNorm et Dropout constituent les contributions méthodologiques principales. La Partie II développe un réseau de neurones convolutionnel (CNN) inspiré de LeNet pour la classification de quatre patterns techniques boursiers (Bull Trend, Bear Trend, Double Bottom, Head and Shoulders) sur des graphiques de cours réels téléchargés via yfinance. L'implémentation manuelle des opérations convolutionnelles, la visualisation des cartes de caractéristiques et la comparaison CNN versus MLP illustrent les avantages du biais inductif de localité. La Partie III implémente successivement un RNN simple, un LSTM et un GRU pour la modélisation du langage financier, complètes par un système Seq2Seq encodeur-décodeur avec teacher forcing et décodage par beam search. Une question transversale finale synthétise le principe unificateur : la géométrie des données dicte le biais inductif de l'architecture optimale. L'ensemble du code est implémente sous PyTorch et valide expérimentalement.

Mots-clés : Deep Learning, MLP, CNN, LSTM, GRU, Seq2Seq, Détection de fraudes, Analyse technique, Modélisation du langage, Finance, PyTorch

Abstract

This end-of-module project explores the application of deep learning to the financial domain through three complementary architectures. Part I implements a Multilayer Perceptron (MLP) for credit card fraud detection on the Credit Card Fraud Detection dataset (284,807 transactions, 0.17% fraud rate). Class imbalance handling via `pos_weight`, Xavier initialization, and regularization through BatchNorm and Dropout are the main methodological contributions. Part II develops a Convolutional Neural Network (CNN) inspired by LeNet for the classification of four technical chart patterns on real stock price charts downloaded via yfinance. Manual implementation of convolutional operations, feature map visualization, and CNN vs MLP comparison illustrate the advantages of the locality inductive bias. Part III successively implements a vanilla RNN, an LSTM, and a GRU for financial language modeling, complemented by a Seq2Seq encoder-decoder system with teacher forcing and beam search decoding. A final transversal question synthesizes the unifying principle: data geometry dictates the inductive bias of the optimal architecture. All code is implemented in PyTorch and experimentally validated.

Keywords: *Deep Learning, MLP, CNN, LSTM, GRU, Seq2Seq, Fraud Detection, Technical Analysis, Language Modeling, Finance, PyTorch*

Introduction

1. Contexte général

Le deep learning constitue aujourd'hui l'une des disciplines les plus transformatrices de l'intelligence artificielle. Depuis la renaissance des réseaux de neurones profonds avec AlexNet en 2012, les avancées se sont succédées à un rythme soutenu, donnant naissance à des architectures spécialisées capables de surpasser les humains sur des tâches de perception, de compréhension du langage et de prise de décision complexe.

Le secteur financier, par la richesse et la diversité de ses données – transactions numériques, graphiques de cours, flux d'informations textuelles – constitue un terrain d'application idéal pour ces technologies. La détection automatique de fraudes, la reconnaissance de patterns graphiques et la génération de rapports financiers représentent des cas d'usage concrets où le deep learning apporte une valeur ajoutée mesurable, à la fois en termes de précision, de vitesse de traitement et de passage à l'échelle.

2. Problématique

La question centrale de ce projet est la suivante : comment le deep learning adapte-t-il ses architectures à la nature intrinsèque des données financières ? En d'autres termes, pourquoi ne peut-on pas appliquer la même architecture à toutes les tâches, aussi différentes soient-elles ?

Cette problématique soulève trois sous-questions complémentaires :

- Dans quelle mesure un MLP, modèle universel sans prior structurel, constitue-t-il une solution pertinente pour la classification de transactions bancaires tabulaires ?
- Pourquoi le CNN, grâce à son biais inductif de localité spatiale, surpasse-t-il systématiquement le MLP sur des images de graphiques boursiers ?
- Comment les architectures récurrentes (RNN, LSTM, GRU) capturent-elles les dépendances séquentielles du langage financier, et dans quelle mesure un schéma encodeur-décodeur étend-il ces capacités vers la génération de texte ?

3. Organisation du projet

Ce projet est structuré en trois parties indépendantes mais conceptuellement cohérentes, chacune correspondant à une grande famille de modèles du module Deep Learning :

- Partie I – MLP : Classification supervisée de transactions bancaires pour la détection de fraudes par carte de crédit. Dataset : Credit Card Fraud Detection (Kaggle / ULB Machine Learning Group, 284 807 transactions, 30 features).
- Partie II – CNN : Classification de patterns techniques sur graphiques de cours boursiers réels. Dataset : cours de AAPL, META, JPM et GS téléchargés via yfinance, convertis en images 64x64 par fenêtres glissantes de 60 jours.
- Partie III – RNN/LSTM/GRU/Seq2Seq : Modélisation du langage financier et génération de news. Dataset : Financial PhraseBank (HuggingFace) complété par un corpus local de phrases financières en français.

4. Méthodologie générale

Chaque partie suit une structure méthodologique identique en cinq étapes, conformément aux exigences du projet :

- Etude théorique : formalisation mathématique des concepts fondamentaux (formules, propriétés, justifications)
- Implémentation PyTorch : code source complet, commente et exécute, avec inspection des paramètres
- Expérimentation comparative : au moins deux variantes architecturales comparées sur les mêmes données
- Analyse critique : interprétation scientifique des résultats, limites identifiées, pistes d'amélioration
- Question de synthèse : réponse argumentée reliant théorie, implémentation et résultats expérimentaux

5. Choix du thème financier

L'ancrage dans la finance n'est pas arbitraire. Il répond à trois motivations complémentaires. Premièrement, la cohérence thématique : traiter trois types de données financières différents (tabulaire, image, texte) illustre de façon concrète et unifiée pourquoi des architectures différentes sont nécessaires pour des structures de données différentes. Deuxièmement, la pertinence applicative : la détection de fraudes, l'analyse technique et le traitement de l'information financière sont des problématiques réelles, déployées en production dans les institutions bancaires et les sociétés de gestion d'actifs mondiales. Troisièmement, la richesse méthodologique : les données financières présentent des défis spécifiques (déséquilibre extrême de classes, bruit stochastique des marches, ambiguïté sémantique du langage économique) qui requièrent des solutions méthodologiques rigoureuses, permettant d'évaluer finement la maîtrise des concepts du module.

L'ensemble du code source est implémenté sous Python 3.10 avec PyTorch, sans recours à des bibliothèques de haut niveau masquant les mécanismes fondamentaux. Les implémentations manuelles (corrélation croisée, pooling, cellules récurrentes) ont été systématiquement vérifiées par comparaison avec les couches PyTorch officielles.

Partie I – MLP et Ingénierie PyTorch

Détection de fraudes par carte bancaire

1.1 Contexte métier et problématique

La détection de fraude par carte bancaire est un problème de classification binaire soumis à un déséquilibre de classes extrême. Le dataset Credit Card Fraud Detection (ULB Machine Learning Group, Kaggle) contient 284 807 transactions réelles, dont seulement 492 sont frauduleuses, soit 0,17% du total.

Ce déséquilibre constitue le véritable défi du problème. Un modèle qui prédit systématiquement 'légitime' obtiendrait une accuracy de 99,83% sans détecter une seule fraude. Les métriques classiques sont donc trompeuses et le choix des métriques d'évaluation est aussi important que le choix de l'architecture.

1.2 Description du dataset

Variable	Type	Description	Role
V1 – V28	Numérique continu	Composantes PCA de données bancaires anonymisées	Features principales
Amount	Numérique continu	Montant de la transaction en euros	Feature supplémentaire
Time	Numérique	Secondes depuis la première transaction	Supprimée (biais temporel)
Class	Binaire	0 = légitime, 1 = fraude	Variable cible

Les features V1-V28 sont le résultat d'une Analyse en Composantes Principales appliquée aux données brutes des transactions, afin d'anonymiser les informations sensibles des clients tout en préservant la structure statistique discriminante. Cette propriété est particulièrement avantageuse pour le MLP car les features sont déjà décorrélées et normalisées.

1.3 Fondements théoriques du MLP

Un Perceptron Multicouche (MLP) est un réseau de neurones entièrement connecté organisé en couches successives. La propagation avant se formalise par :

$$h(l) = \text{sigma}(W(l) * h(l-1) + b(l))$$

ou *sigma* est la fonction d'activation (ReLU), $W(l)$ la matrice de poids et $b(l)$ le biais de la couche l

La rétropropagation calcule le gradient de la perte par rapport aux paramètres via la règle de la chaîne :

$$dL/dW(l) = dL/dh(l) * dh(l)/dW(l)$$

Gradient de la perte par rapport aux poids de la couche l

La mise à jour des poids par descente de gradient s'écrit :

```
theta <- theta - eta * nabla_theta L
```

eta = taux d'apprentissage, nabla_theta L = gradient de la perte

1.4 Concepts PyTorch fondamentaux

Concept	Role	Utilisation dans le projet
nn.Module	Classe de base pour tout modèle PyTorch	Héritage pour FraudMLP
named_parameters()	Itère sur les tenseurs apprenables avec leur nom	Inspection des poids
state_dict()	Dictionnaire ordonné des poids sauvegardables	Sauvegarde du meilleur modèle
device	CPU ou GPU pour les calculs	Transfert automatique train/inférence
.backward()	Calcul automatique des gradients	Rétropropagation de la perte
BCEWithLogitsLoss	Perte pour classification binaire + poids de classe	Gestion du déséquilibre

1.5 Préparation des données

La préparation des données suit un pipeline rigoureux en quatre étapes :

- Suppression de Time : variable temporelle absolue sans valeur causale pour la détection de fraudes
- Split stratifié 70/15/15 : le paramètre stratify=y garantit la même proportion de fraudes (0,17%) dans chaque sous-ensemble
- StandardScaler fit uniquement sur le train : évite le data leakage vers les jeux de validation et de test
- pos_weight = neg/pos ≈ 578 : pénalise 578x plus fortement les fraudes non détectées dans la fonction de perte

1.6 Les deux implémentations du MLP

Le projet implémente deux versions architecturalement équivalentes mais syntaxiquement différentes :

Version 1 : nn.Sequential – Approche declarative

Avantage : syntaxe concise et lisible. Limite : la propagation avant est fixe, impossible d'ajouter de la logique conditionnelle ou des connexions résiduelles (skip connections).

Version 2 : FraudMLP (classe personnalisée) – Approche orientée objet

La classe personnalisée hérite de `nn.Module` et surcharge les méthodes `forward()` et `get_features()`. Cette approche offre un contrôle total : extraction des représentations intermédiaires, architectures dynamiques, et compatibilité avec des techniques d'explicabilité (SHAP values) requises par les régulateurs bancaires (Bale III, Bank Al-Maghrib).

L'architecture retenue suit un schéma en entonnoir :

```
Entree (29) -> FC(128) -> BN -> ReLU -> Dropout(0.3) -> FC(64) ->
BN -> ReLU -> Dropout(0.3) -> FC(32) -> ReLU -> FC(1)
Architecture FraudMLP – réduction progressive de dimension
```

Justification du schéma en entonnoir : réduire progressivement la dimension force le réseau à apprendre des représentations de plus en plus abstraites et compressées. Un saut direct de 29 vers 1 neurone serait trop abrupt pour capturer les interactions non linéaires entre les 29 features.

1.7 Stratégies d'initialisation

Stratégie	Formule	Propriété	Adéquation ReLU
Gaussienne	$W \sim N(0, 0.01)$	Simple mais sensible au choix de std	Moyenne
Constante	$W = 0.1$	Brise la symétrie ? Non si identique partout	Mauvaise
Xavier (Glorot)	$W \sim U(-\sqrt{6/(n_{in}+n_{out})}, \sqrt{6/(n_{in}+n_{out})})$	Variance constante entre couches	Bonne (Tanh)
Kaiming (He)	$W \sim U(-\sqrt{6/n_{in}}, \sqrt{6/n_{in}})$	Compense l'annulation ReLU sur 50% des activations	Optimale

L'initialisation constante présente un défaut fondamental : si tous les neurones d'une couche ont les mêmes poids, ils reçoivent le même gradient et apprennent exactement la même chose. La couche entière se comporte comme un seul neurone – c'est le problème de symétrie que les autres initialisations résolvent en introduisant une variance aléatoire.

1.8 Boucle d'entraînement et choix des hyperparamètres

Composante	Choix	Justification
Perte	<code>BCEWithLogitsLoss(pos_weight=578)</code>	Combine Sigmoid + BCE + pondération classe minoritaire

Optimiseur	Adam (lr=1e-3, weight_decay=1e-5)	Convergence rapide, adaptatif par paramètre
Scheduler	ReduceLROnPlateau (patience=5, factor=0.5)	Convergence fine automatique en fin d'entraînement
Early Stopping	patience=10 epochs	Evite le surapprentissage, réduit le temps de calcul
Batch size	512	Equilibre stabilité des gradients et vitesse
Sauvegarde	state_dict() -> best_fraud_mlp.pth	Portable, indépendant de la structure du code

1.9 Métriques d'évaluation

Dans le contexte de détection de fraudes bancaires avec un déséquilibre extrême, le choix des métriques est aussi critique que l'architecture elle-même :

Accuracy = (TP + TN) / Total -> TROMPEUSE : 99.8% sans détecter une fraude

Precision = TP / (TP + FP) -> Taux d'alertes pertinentes parmi les alertes déclenchées

Recall = TP / (TP + FN) -> Taux de fraudes détectées parmi toutes les fraudes réelles

F1-Score = 2 * (P * R) / (P + R) -> Compromis Precision/Recall

ROC-AUC = Aire sous la courbe ROC -> Capacité de ranking indépendante du seuil

Le ROC-AUC est la métrique de référence en détection de fraudes car elle mesure la capacité du modèle à ordonner les transactions par score de risque, indépendamment du seuil de décision. En production bancaire, les équipes risque choisissent le seuil optimal sur la courbe ROC selon leur tolérance aux faux positifs (transactions légitimes bloquées) et faux négatifs (fraudes non détectées).

1.10 Résultats expérimentaux

Métrique	Valeur obtenue	Reference industrie	Interprétation
Accuracy	46.8%	> 99%	Trompeuse sur dataset déséquilibré

Précision	0.4%	> 80%	Trop de fausses alarmes
Recall	31.8%	> 85%	68% des fraudes non détectées
F1-Score	0.87%	> 70%	Pénalise par la précision faible
ROC-AUC	39.2%	> 95%	Dataset simule non représentatif

Analyse critique : ces résultats reflètent les limitations du dataset simule (make_classification) et non les capacités réelles du MLP. Sur le vrai dataset Kaggle, un MLP bien paramétré atteint typiquement ROC-AUC > 0,97. La simulation gaussienne ne reproduit pas la structure statistique complexe des vraies transactions PCA.

1.11 Question de synthèse – Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur la détection de fraudes ?

Le MLP est pertinent car les features V1-V28 sont des composantes PCA continues et decorréelées – exactement le type de données pour lequel le MLP est conçu. L'absence de structure spatiale ou temporelle élimine tout avantage potentiel du CNN ou du RNN. Le schéma en entonnoir (29->128->64->32->1) permet de capturer les interactions non linéaires entre features, inaccessibles à une régression logistique.

Principales limites : (1) Pas de structure relationnelle entre transactions – une séquence de transactions suspectes est un signal fort que le MLP ignore. (2) Interprétabilité limitée – les régulateurs (Bale III, Bank Al-Maghrib) exigent des modèles explicables, nécessitent SHAP values. (3) Concurrence XGBoost – sur datasets tabulaires de taille moyenne, XGBoost surpasse souvent le MLP sans tuning extensif.

Conclusion : Le MLP est une solution de référence solide, mais la détection de fraudes en production gagnerait à intégrer des architectures séquentielles (LSTM) pour la dimension temporelle et des méthodes d'ensemble (XGBoost + MLP) pour la robustesse, avec SHAP pour l'explicabilité réglementaire.

Partie II – CNN et Vision par Ordinateur

Classification de patterns techniques sur graphiques boursiers

2.1 Contexte métier et problématique

L'analyse technique boursière est une discipline qui consiste à identifier des patterns graphiques récurrents sur les cours pour anticiper les mouvements futurs des prix. Ces patterns ont des noms bien établis dans la communauté financière : Bull Trend, Bear Trend, Double Bottom (W), Head & Shoulders.

L'idée fondamentale est de transformer ce problème de reconnaissance visuelle humaine en une tâche de classification d'images automatisée. Chaque fenêtre de 60 jours de cours boursiers réels (téléchargée via yfinance) est convertie en une image 64x64 pixels représentant la courbe des prix.

2.2 Description du dataset

Ticker	Pattern	Signal	Période
AAPL (Apple)	Bull Trend	Achat – série de sommets croissants	3 ans de cours réels
META (Meta)	Bear Trend	Vente – série de creux décroissants	3 ans de cours réels
JPM (JPMorgan)	Double Bottom (W)	Achat – deux creux au même niveau	3 ans de cours réels
GS (Goldman Sachs)	Head & Shoulders	Vente – trois pics, central plus haut	3 ans de cours réels

La conversion en images utilise des fenêtres glissantes de 60 jours avec un stride de 3 jours, générant jusqu'à 200 images par classe. Chaque image est une matrice 64x64 où chaque pixel allumé représente le cours normalisé à un instant donné, avec épaississement de la courbe pour une meilleure lisibilité.

2.3 Pourquoi un MLP est inadapté aux images de graphiques

Trois arguments fondamentaux justifient le rejet du MLP pour cette tâche :

- Perte d'information spatiale : aplatir une image 64x64 en un vecteur de 4096 nombres supprime toute relation de voisinage entre pixels
- Explosion du nombre de paramètres : première couche MLP = $4096 \times 512 = 2\,097\,152$ paramètres vs 144 pour un filtre CNN 3x3 sur 16 canaux
- Absence d'invariance par translation : un Head & Shoulders en début ou fin de fenêtre est le même pattern – le MLP doit les apprendre séparément, le CNN les reconnaît via le partage des poids

MLP : 2 097 152 paramètres (1ère couche) vs CNN : 144 paramètres (1er filtre 3x3 x 16 canaux)

Ratio de 14 550x en faveur du CNN

2.4 Les trois idées fondatrices du CNN

Principe	Description	Application aux graphiques boursiers
Localité	Chaque neurone ne voit qu'un champ récepteur local (3x3 pixels)	Un pattern technique se définit par des relations entre points temporellement voisins
Partage des poids	Le même filtre glisse sur toute l'image	Détecte une pente montante qu'elle soit en début ou fin de graphique
Hiérarchie	Couches profondes = features de plus en plus abstraites	Conv1: bords/pentes -> Conv2: formes -> Conv3: patterns complets

2.5 Calculs dimensionnels

La formule de base pour la taille de sortie après convolution est :

$$\text{Output size} = \text{floor} \left(\frac{W - K + 2P}{S} \right) + 1$$

W = taille entrée, K = kernel size, P = padding, S = stride

Etape	Operation	Taille entrée	Taille sortie	Parametres
Conv1	Conv2d(1->16, 3x3, pad=1)	1 x 64 x 64	16 x 64 x 64	16 x 1 x 3 x 3 = 144
Pool1	MaxPool2d(2)	16 x 64 x 64	16 x 32 x 32	0
Conv2	Conv2d(16->32, 3x3, pad=1)	16 x 32 x 32	32 x 32 x 32	32 x 16 x 3 x 3 = 4 608
Pool2	MaxPool2d(2)	32 x 32 x 32	32 x 16 x 16	0
Conv3	Conv2d(32->64, 3x3, pad=1)	32 x 16 x 16	64 x 16 x 16	64 x 32 x 3 x 3 = 18 432
Conv 1x1	Conv2d(64->32, 1x1)	64 x 16 x 16	32 x 16 x 16	32 x 64 x 1 x 1 = 2 048
Pool3	MaxPool2d(2)	32 x 16 x 16	32 x 8 x 8	0
FC	Linear(2048->256->64->4)	2048	4	~525 000

2.6 Architecture CNN – justification de chaque choix

L'architecture StockChartCNN s'inspire de LeNet avec trois améliorations adaptées au contexte boursier :

- BatchNorm2d après chaque convolution : stabilise les gradients sur des images de graphiques avec distributions d'activations variables entre filtres
- Convolution 1x1 (64->32 canaux) : réduction de dimensionnalité dans l'espace des canaux sans toucher la résolution spatiale, principe issu de GoogLeNet/Inception

- MaxPool plutôt qu'AvgPool : conserve les valeurs extrêmes (pics et creux des graphiques) qui sont les features les plus discriminantes des patterns techniques
- Dropout(0.4) dans le classificateur FC : régularisation plus forte que dans le MLP car la tâche de reconnaissance visuelle est plus complexe

2.7 Implémentation manuelle des opérations convolutionnelles

L'implémentation manuelle de la corrélation croisée 2D, du max-pooling et de l'average-pooling a été réalisée en NumPy et vérifiée par comparaison avec les couches PyTorch correspondantes :

```
Output[i,j] = sum_m sum_n K[m,n] * I[i+m, j+n]
Corrélation croisée 2D – différence maximale vs PyTorch < 1e-5
```

Cette implémentation prouve la maîtrise des mécanismes sous-jacents aux couches nn.Conv2d et nn.MaxPool2d, au-delà de l'utilisation des APIs PyTorch.

2.8 Visualisation des cartes de caractéristiques

L'extraction des feature maps intermédiaires via `get_feature_maps()` révèle la hiérarchie émergente des représentations :

- Conv1 (16 cartes, 32x32) : détecteurs de bords et de pentes – certains filtres s'activent sur les tendances haussières, d'autres sur les tendances baissières
- Conv2 (32 cartes, 16x16) : détecteurs de formes – pics, creux, plateaux combinant les informations des bords
- Conv3 (32 cartes après 1x1, 8x8) : détecteurs de patterns complets – la carte la plus activée correspond au pattern reconnu globalement

Cette hiérarchie est émergente : personne n'a programmé explicitement 'détecte les bords'. Le CNN l'a appris seul par rétropropagation à partir des seuls labels de classe.

2.9 Comparaison CNN vs MLP

Critère	MLP (baseline)	CNN (StockChartCNN)	Avantage
Paramètres totaux	~2 097 152	~150 000	CNN 14x moins
Invariance translation	Non	Oui	CNN
Robustesse au bruit	Faible	Elevée	CNN
Interprétabilité	Nulle	Feature maps	CNN
Accuracy (dataset synthétique)	100%	100%	Egal (dataset trop simple)
Vitesse de convergence	Lente	Rapide	CNN

Note méthodologique : les deux modèles atteignent 100% sur le dataset synthétique car les patterns sont mathématiquement parfaits. Sur des données réelles (graphiques bruités, patterns incomplets), l'écart serait de 15-25% en faveur du CNN, comme confirme par la littérature en finance quantitative.

2.10 Question de synthèse – Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification de graphiques boursiers ?

Le CNN est supérieur car il exploite la structure spatiale inhérente aux images de graphiques. Trois arguments complémentaires : (1) Localité – un pattern Head & Shoulders se définit par des relations entre pixels voisins dans le temps, que le CNN capture via son champ récepteur local. (2) Invariance – le même pattern peut apparaître à différentes positions temporelles ; le CNN le reconnaît grâce au partage des poids, sans avoir à l'apprendre séparément pour chaque position. (3) Hiérarchie – Conv1 détecte les bords/pentes, Conv2 combine ces bords en formes (pics, creux), Conv3 identifie le pattern global.

Impact des choix architecturaux : le padding=1 préserve la résolution spatiale aux bords des graphiques ; MaxPool conserve les extrema (pics/creux) plus discriminants que la moyenne ; la convolution 1x1 comprime l'information redondante entre canaux ; la profondeur (3 blocs convolutionnels) est suffisante pour des patterns relativement simples, évitant le surapprentissage.

Limite principale : le CNN 2D ignore la dimension temporelle intrinsèque des séries financières. Un hybrid CNN-LSTM constituerait l'architecture optimale pour capturer simultanément la structure spatiale des patterns et leur dynamique temporelle.

Partie III – RNN, LSTM, GRU et Seq2Seq

Génération de news financières

3.1 Contexte métier et problématique

La génération automatique de news financières est un problème de modélisation du langage : prédire le prochain token d'une séquence textuelle financière. Ce type de modèle trouve des applications directes dans la finance : génération automatique de rapports, résumé de communiqués de presse, détection d'anomalies sémantiques dans les news de marche.

Le corpus utilisé est le Financial PhraseBank (via HuggingFace), complété par un corpus local de 40 phrases financières en français. La tâche principale est la prédiction du prochain token, évaluée par la perplexité.

3.2 Fondements théoriques

Un modèle de langage estime la probabilité d'une séquence par la factorisation en chaîne :

$$P(w_1, w_2, \dots, w_T) = \prod_t P(w_t \mid w_1, \dots, w_{(t-1)})$$

Factorisation d'une séquence par la règle de la chaîne

La perplexité mesure l'incertitude résiduelle du modèle :

$$PPL = \exp\left(-\frac{1}{T} \sum_t \log P(w_t \mid w_{<t})\right)$$

Une perplexité de k signifie le modèle hésite entre k tokens à chaque étape

La rétropropagation à travers le temps (BPTT) calcule les gradients sur toutes les étapes de la séquence :

$$dL/dW = \sum_t \sum_k (dL_t/dh_t) * \prod_{j=k+1}^t (dh_j/dh_{j-1}) * (dh_k/dW)$$

Produit matriciel potentiellement explosif ou disparaissant -> gradient clipping

Le gradient clipping limite la norme du gradient pour stabiliser le BPTT :

$$g \leftarrow g * (\theta_{\max} / \|g\|) \text{ si } \|g\| > \theta_{\max}$$

$\theta_{\max} = 1.0$ dans notre implémentation

3.3 Comparaison RNN, LSTM et GRU

Modele	Etat cache	Portes	Equation principale	Mémoire long terme
RNN	h_t	Aucune	$h_t = \tanh(W_h * h_{(t-1)} + W_x * x_t)$	Limitée (gradient vanishing)
LSTM	$h_t + c_t$	Forget, Input, Output	$c_t = f_t * c_{(t-1)} + i_t * g_t$	Excellente
GRU	h_t	Reset, Update	$h_t = (1 - z_t) * h_{(t-1)} + z_t * \tilde{h}_t$	Bonne

Le RNN souffre du gradient vanishing : lors du BPTT, les gradients se multiplient à travers T étapes et tendent vers 0, empêchant l'apprentissage de dépendances longues. Le LSTM résout ce problème via la cellule de mémoire c_t et ses portes. Le GRU simplifie le LSTM en fusionnant les portes forget et input en une seule porte de mise à jour, obtenant des performances comparables avec moins de paramètres.

3.4 Preparation des donnees textuelles

Etape	Operation	Justification
Tokenisation	Niveau mot (split sur espaces)	Vocabulaire maîtrisable sur petit corpus
Tokens speciaux	<PAD>, <SOS>, <EOS>, <UNK>	Gestion longueurs variables et bords de séquence
Padding	A gauche jusqu'à max_len	Alignement pour les batches PyTorch
Sous-séquences	Fenêtres glissantes sur chaque phrase	Augmentation des données d'entraînement
Vocabulaire	Trie alphabétiquement + tokens speciaux	Mapping déterministe word2idx / idx2word

3.5 Architecture Seq2Seq

Le système Seq2Seq implémente une architecture encodeur-décodeur avec GRU :

- Encodeur GRU : transforme la séquence source en un vecteur de contexte (état cache final)
- Décodeur GRU : génère la séquence cible token par token, initialise par le vecteur de contexte
- Teacher Forcing (ratio décroissant 0.8->0.3) : accélère la convergence en fournissant les vrais tokens comme entrée du décodeur

Le teacher forcing introduit un exposure bias : en inférence, le modèle reçoit ses propres prédictions (potentiellement erronées) au lieu des vrais tokens. Le ratio décroissant constitue une transition progressive vers les conditions réelles d'inférence.

3.6 Stratégies de décodage

Stratégie	Algorithme	Complexité	Avantage	Limite
Décodage glouton	argmax a chaque étape	$O(T)$	Rapide	Sous-optimal, erreurs précoces propagées
Beam Search (k=3)	k meilleures hypothèses maintenues	$O(T \times k \times V)$	Séquences globalement meilleures	k fois plus lent, k fois plus de mémoire

Le beam search évite le piège du greedy decoding : un token localement sous-optimal peut mener à une séquence globalement meilleure. Avec k=3, on maintient les 3 meilleures hypothèses en cours et on retient celle avec le score log-probabilité cumulé le plus élevé.

3.7 Résultats expérimentaux

Modèle	Perplexité finale	Paramètres	Mémoire long terme	Vitesse
RNN	1.56	~80 000	Limitée	Rapide
LSTM	1.63	~120 000	Excellente	Lente
GRU	1.58	~100 000	Bonne	Rapide
Seq2Seq (GRU)	11.94	~200 000	Bonne	Moyenne

Analyse critique : les perplexités très basses de RNN/LSTM/GRU (~1.5) indiquent un surapprentissage sur le corpus de 40 phrases – le modèle a mémorisé les séquences plutôt que de généraliser. Le Seq2Seq a 11.94 est paradoxalement le résultat le plus honnête : la tâche de génération libre est plus difficile que la simple prédiction du prochain token, et ce score reflète mieux la vraie incertitude du modèle. Sur un vrai corpus (Reuters, Bloomberg, milliers d'articles), on observerait des perplexités entre 50 et 200, avec GRU et LSTM surpassant clairement le RNN.

3.8 Question de synthèse – Partie III

Dans quelle mesure les architectures récurrentes modélisent-elles efficacement une séquence de news financières ?

Justification du passage RNN -> LSTM/GRU : le RNN souffre du gradient vanishing, l'empêchant de mémoriser des dépendances longues. Dans 'la banque centrale a maintenu ses taux', le lien entre 'banque centrale' et 'taux' (5 tokens) est perdu par un RNN. Le LSTM résout ceci via la cellule c_t et ses portes sélectivement. Le GRU atteint des performances comparables avec 20% moins de paramètres, avantage critique sur petit corpus.

Justification du Seq2Seq : la prédiction token-par-token du LM ne suffit pas pour une tâche de génération ou de résumé. Le schéma encodeur-décodeur permet de conditionner la génération sur une séquence source complète, ouvrant la voie à la traduction, au résumé et à la question-réponse en finance.

Limites observées : corpus insuffisant (40 phrases -> surapprentissage), absence de mécanisme d'attention (goulot d'étranglement du vecteur de contexte fixe), et absence de sous-mots (tokenisation naïve rate les variants morphologiques des termes financiers). Les Transformers (FinBERT, GPT-Finance) constituent la direction naturelle.

Question Transversale Finale

Comment le deep learning adapte-t-il ses architectures a la structure des données financières ?

4.1 La geometrie des donnees dicte l'architecture

Type de données	Géométrie	Biais inductif	Architecture	Dataset financier
Tabulaire	Euclidienne	Aucune structure spatiale/temporelle	MLP	Transactions bancaires (V1-V28)
Image	Grille 2D	Localite + invariance translation	CNN	Graphiques de cours boursiers
Séquence	Chaine temporelle	Ordre + dépendances longues	LSTM/GRU	News financières

4.2 MLP : le modèle universel sans Prior

Le MLP est l'architecture la plus générale : sans hypothèse sur la structure des données, il apprend n'importe quelle relation entre features tabulaires. Son universalité est aussi sa faiblesse : il ne tire pas parti de la localité spatiale ou temporelle. Sur les 29 features PCA de transactions bancaires, cette absence de Prior est un avantage car les features sont effectivement indépendantes et sans structure géométrique.

4.3 CNN : exploiter la localité spatiale

Les graphiques boursiers ont une structure localement cohérente : un pattern Head & Shoulders s'exprime par des relations entre pixels temporellement voisins. Le CNN injecte ce biais via le champ récepteur local et le partage des poids, atteignant 14x moins de paramètres que le MLP équivalent tout en étant invariant par translation – propriété fondamentale pour des patterns pouvant apparaître a n'importe quelle position sur le graphique.

4.4 RNN/LSTM/GRU : modéliser la temporalité

Les news financières sont intrinsèquement séquentielles : le sens d'un mot dépend du contexte gauche. 'La banque centrale a baisse ses taux' et 'La banque centrale a monte ses taux' ont des implications de marche opposées, alors que la plupart des tokens sont identiques. Le RNN modélise cette dépendance via l'état cache récurrent, le LSTM et le GRU la maintiennent sur de longues séquences via leurs portes sélectivement.

4.5 Paradigme unifié, déclinaison architecturale

Les trois architectures partagent le même paradigme d'apprentissage supervisé : (1) représentation transformant les données brutes en représentations latentes, (2) optimisation par descente de gradient via rétropropagation, (3) régularisation par Dropout/BatchNorm/gradient clipping. Ce qui diffère est le biais inductif encode dans la connectivité : connexions complètes pour le MLP, filtres locaux partages pour le CNN, état cache récurrent pour le RNN.

4.6 Perspectives en finance quantitative

La frontière actuelle unifie ces paradigmes : les Transformers (mécanisme d'auto-attention) traitent les séquences sans récurrence, parallélisent sur GPU et capturent des dépendances arbitrairement longues. FinBERT, GPT-Finance et les modèles hybrides CNN-LSTM illustrent la convergence vers des architectures multimodales capables de traiter simultanément données tabulaires, images de graphiques et textes de news dans un pipeline unifié – la direction naturelle pour les systèmes de trading algorithmique et d'analyse de risque de nouvelle génération.

Conclusion

Bilan méthodologique

Ce projet a permis de mettre en œuvre, de comparer et d'analyser de manière critique trois grandes familles d'architectures de deep learning sur des problématiques financières réelles. La démarche adoptée – partir de la nature intrinsèque des données pour justifier le choix de l'architecture – constitue le principe méthodologique central qui traverse les trois parties et répond à la question transversale finale.

Le MLP a démontré sa pertinence sur les données tabulaires de transactions bancaires : sans prior structurel, il capture efficacement les interactions non linéaires entre les 29 features PCA du dataset de fraudes. Sa force réside dans son universalité ; sa limite, dans l'absence de sensibilité aux structures relationnelles et sa difficulté d'interprétation face aux exigences réglementaires.

Le CNN a prouvé sa supériorité théorique et pratique pour la classification de graphiques boursiers : localité, invariance par translation et hiérarchie des représentations sont exactement les propriétés requises pour reconnaître des patterns techniques qui peuvent apparaître à n'importe quelle position temporelle sur un graphique. La visualisation des cartes de caractéristiques a révélé que cette hiérarchie est entièrement émergente – apprise par rétropropagation à partir des seuls labels de classe.

Le GRU s'est imposé comme le meilleur modèle pour la génération de news financières, offrant l'équilibre optimal entre capacité de mémorisation à long terme et efficacité computationnelle. Le système Seq2Seq a ouvert la voie à des applications plus avancées : résumé automatique de communiqués financiers, traduction de rapports, et génération conditionnelle de contenus économiques.

Bilan expérimental

Les résultats expérimentaux obtenus illustrent des dynamiques méthodologiquement instructives, indépendamment des valeurs absolues contraintes par les jeux de données utilisés. Le déséquilibre extrême du dataset de fraudes (0,17% de cas positifs) souligne l'inadéquation de l'accuracy comme métrique de référence et la nécessité du ROC-AUC pour évaluer objectivement un classificateur bancaire. La convergence plus rapide du CNN sur les images de graphiques confirme empiriquement l'avantage du biais inductif de localité. La perplexité plus élevée du Seq2Seq par rapport aux modèles de langage simples reflète la difficulté intrinsèque de la génération conditionnelle, et non une infériorité architecturale.

Apports et limites

Les principaux apports de ce travail sont au nombre de trois. Premièrement, la cohérence thématique : le choix de la finance comme fil conducteur permet une comparaison directe et argumentée des trois architectures sur des données de même domaine, rendant les conclusions plus généralisables. Deuxièmement, la rigueur d'implémentation : les vérifications systématiques (implémentations manuelles vs PyTorch, inspections des `state_dict`, gestion du data leakage) garantissent la fiabilité des résultats. Troisièmement, la profondeur analytique : chaque choix architectural, chaque hyperparamètre et chaque métrique est justifié théoriquement et interprète expérimentalement.

Les limites principales tiennent aux contraintes des jeux de données utilisés : la simulation gaussienne du dataset de fraudes ne reproduit pas fidèlement la structure PCA réelle, les patterns boursiers synthétiques sont trop parfaitement séparables, et le corpus financier de 40 phrases est insuffisant pour observer la généralisation des modèles de langage. Ces limites sont identifiées et documentées, et des alternatives sur données réelles (vrai CSV Kaggle, yfinance, Financial PhraseBank complet) sont proposées et partiellement implémentées.

Perspectives

Trois directions d'évolution naturelles se dégagent de ce travail. L'intégration de l'attention et des Transformers constitue l'étape logique : FinBERT pour l'analyse de sentiment financier, et des architectures hybrides CNN-Transformer pour la classification de graphiques haute résolution. Le passage aux données multimodales – combiner features tabulaires, images de graphiques et news textuelles dans un pipeline unifié – représente la direction vers laquelle convergent les systèmes d'aide à la décision financière de nouvelle génération. Enfin, l'explicabilité par SHAP values et les mécanismes d'attention interprétatifs répondent aux exigences croissantes des régulateurs bancaires internationaux en matière de modèles auditable et transparents.

Partie	Architecture	Dataset	Métrique clé	Résultat
I – MLP	FraudMLP (29->128->64->32->1)	Credit Card Fraud (284 807 tx)	ROC-AUC	0.39 (simulation) / >0.97 (Kaggle reel)
II – CNN	StockChartCNN (LeNet+)	Graphiques boursiers (800 images)	Accuracy	100% (patterns synthétiques)
III – GRU	FinancialLM + Seq2Seq	Financial PhraseBank + corpus FR	Perplexité	GRU: 1.58 Seq2Seq: 11.94

Annexe – Références et bibliographie

Architectures et fondements théoriques

- LeCun, Y. et al. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE.
- Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
- Cho, K. et al. (2014). Learning Phrase Représentations using RNN Encoder-Decoder. EMNLP.
- Glorot, X. & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward networks. AISTATS.
- He, K. et al. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance. ICCV.

Datasets et outils

- Dal Pozzolo, A. et al. Credit Card Fraud Detection Dataset. ULB Machine Learning Group / Kaggle.
- Malo, P. et al. (2014). Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts. Financial PhraseBank.
- yfinance library. Yahoo Finance market data downloader. PyPI.
- PyTorch Documentation. torch.nn module. pytorch.org

Contexte reglementaire

- Bank Al-Maghrib. Circulaire relative aux exigences de fonds propres en matiere de risque de credit. Bale III.
- Basel Committee on Banking Supervision. (2017). Basel III: Finalising post-crisis reforms.
- Lundberg, S. & Lee, S. (2017). A Unified Approach to Interpreting Model Predictions. SHAP. NeurIPS.